

Senior Analytics Engineer: Take home

Context

We'd like you to work through a solution to solve one product and one operations shaped problem. We've given details on the data and task below. Some tips to help you timebox your work:

- This exercise should take about 3-4 hours. If it takes a lot more, please let us know what you felt was not scoped well and we'll try to adjust the exercise/expectations. We give you 48 hours to complete the exercise.
- While your submission should be easy to read, clear and neat, please don't spend time on presentation. Code, bullet points and diagrams are all welcome. Markdown is encouraged but you're welcome to use whatever medium you're comfortable in. However, we actively discourage powerpoint/google slides or other forms of presentation that require a lot of design investment.
- If something's unclear or could go either way, make a decision and let us know what you assumed and why. We would like you to demonstrate your own judgment as making sense of ambiguity is a big part of this job.

Data

Three core tables are described below. Please take time to think and reason about them. Not strictly necessary but If it's helpful, you can simulate your own data.

Payments

- Magnitude: Billions of rows
- `status` takes on the following values: 'success', 'failed', 'reversed'. When a payment is reversed, a historical `status` is revised. This means that an existing row in the data is updated with the new status ('reversed'). The date of reversal is recorded in the `updated_at` date.
- `currency` takes on ISO format, like "UGX", "XOF", etc.
- `country` takes on ISO format, like "UG", "SN", "CI", etc.
- `channel` takes on the following values: 'ussd', 'app', 'agent'. It indicates the source of a payment.

SQL

```
create or replace table payments (  
  payment_id    varchar      not null,  
  wallet_id     varchar      not null,  
  merchant_id   varchar,  
  amount        number(12,2) not null,  
  currency      varchar(3)   not null,  
  status        varchar      not null,  
  created_at    timestamp_ntz not null,  
  updated_at    timestamp_ntz not null,  
  country       varchar(2)   not null,  
  channel       varchar      not null  
)
```

Merchants

- Magnitude: Thousands of rows
- **status** takes on the following values: 'active', 'inactive', 'suspended'
- **country** takes on ISO format, like "UG", "SN", "CI", etc.
- **category** takes on the following values: 'supermarket', 'ecommerce', 'taxi', 'utility', 'other'

SQL

```
create or replace table merchants (  
  merchant_id    varchar      not null,  
  merchant_name  varchar      not null,  
  category       varchar,  
  country        varchar(2)   not null,  
  status         varchar      not null,  
  created_at     timestamp_ntz not null,  
  updated_at     timestamp_ntz not null  
);
```

Wallets

- Magnitude: Millions of rows

- A wallet is an account held by a Wave customer with Wave. For the purposes of this exercise you can assume that a wave customer only has one wallet.
- **status** takes on the following values: 'active', 'inactive', 'blocked'
- **country** takes on ISO format, like "UG", "SN", "CI", etc.
- **is_agent** takes on the following values: true, false.
 - An agent is an individual that accepts deposits and makes payments to Wave customers.
- **kyc_tier** takes on the following values: 0, 1, 2. Some further notes on KYC tiers:
 - KYC: “Know Your Customer” is a mandatory process where businesses, especially financial institutions, verify the identity of their customers to prevent financial crime such as money laundering and fraud.
 - In most financial ecosystems, there are caps on how much a specific legal entity (a customer) can transact.
 - KYC tiers refers to the level of certainty we’ve assigned to knowing an entity’s identity and therefore what their maximum transaction limit is. The idea is that the higher degree of certainty we have in knowing a customer’s identity, the larger the limit we can grant them. Often these limits are set by external regulatory bodies.
 - For the purposes of this exercise, we assume that a KYC tier can only increase. So an entity can go from 0 to 1 to 2 but never the other way around.
 - When a wallet's **kyc_tier** is updated, a historical **kyc_tier** is revised. This means that an existing row in the data is updated with the new **kyc_tier**. The date of the update is recorded in the **updated_at** date.
 - Tier 0 is the lowest tier and essentially means that the wallet is not KYC'd. This means that the user can only transact up to USD 100 (in equivalent local currency) per month.
 - Tier 1 is the middle tier and requires some KYC. This means that the user can transact up to USD 1,000 (in equivalent local currency) per month.
 - Tier 2 is the highest tier and requires full KYC. This means that the user can transact up to USD 10,000 (in equivalent local currency) per month.

SQL

```
create or replace table wallets (
  wallet_id      varchar      not null,
  user_created_at timestamp_ntz not null,
  kyc_tier       int,
  country        varchar(2)    not null,
  is_agent       boolean       not null,
  status         varchar       not null,
  updated_at     timestamp_ntz not null
);
```

Task

Design a solution that supports the use cases below. The solution should include the following:

- A design document. This is a document that describes your proposed data architecture, alternatives you've explored and possible challenges with your solution.
- DBT models for the first use case. You can also provide DBT models for the second use case but that's optional.
- Feel free to add in whatever you think makes your solution easier to understand/explain (macros, tests, docs, diagrams in mermaid/excalidraw etc).

Use cases

- KPI dashboards that have KPIs listed below. You can assume that the dashboards are written in SQL, have both current and historical values, are refreshed every hour and that performance is valued highly.
 - Total number of payments: Aggregate and breakdown by country, status, channel, merchant category and KYC tier
 - Gross volume: Sum of amount for all non-failed transactions. This includes both successful and reversed transactions.
 - Net volume: Sum of amount for all successful transactions without the reversed amounts.
- Compliance checks that ensure users can only transact within their KYC limit. That is, if the total amount transacted within a calendar month is lower or equal to the limit assigned to the user. A couple of notes:
 - The analyst team has to report on two outputs here:
 - Checks that run every 15 mins and updates a list of breaching users (with transaction details) for the current month.
 - A historical record of all checks run (so we can measure compliance over time).
 - A couple of notes to help build your solution:
 - The **wallets** table only stores the most recent KYC status (and when it was updated).
 - All three tables are updated every 15 minutes. You can assume that they are updated simultaneously and don't have drift between them.